

Assessing the Viability of Pre-Local-Search Surrogate Filtering: A Diagnostic Study of Survival-Based Offspring Pruning in Memetic CETSP

Demir Balemir^{a,*}, Deniz Cantürk^a

^aDepartment of Computer Engineering, TED University, Ankara, Türkiye

ARTICLE INFO

Keywords:

Close-enough traveling salesman problem
Memetic algorithm
Surrogate-assisted evolutionary computation
Survival analysis
Local search
Viability assessment

Abstract

Surrogate-assisted evolutionary computation substitutes cheap learned predictions for expensive fitness evaluations. In memetic algorithms the analogous idea is to filter offspring *before* local search, since local search dominates the per-iteration cost. We realise this idea in full for the Close-Enough Traveling Salesman Problem (CETSP): offspring are described at birth by geometric and cost features, labelled by their survival time in the population, and scored by a survival model that gates variable-neighbourhood descent (VND). Nine survival models run concurrently on parallel islands alongside an in-run no-filter control. On 53 benchmark instances with five seeds per configuration, we detect no statistically significant differences in solution quality or convergence speed. Diagnostic analyses find limited discrimination (cross-validated concordance ≈ 0.53), with similar findings across the tested model classes and feature families. An oracle using recorded post-VND information reaches 0.687, providing an empirical reference for prediction. Stage-wise measurements show instance-dependent rank transmission through greedy, Lin–Kernighan and refinement stages. Lower-ranked inputs tend to improve more. These observations motivate a one-sided viability gate centred on three checks: target predictability, rank transmission and discrimination beyond drift. The diagnostics are conditional on recorded, admitted solutions. Adverse findings can support reconsidering the tested design; favourable findings justify further evaluation without establishing that filtering will pay off.

1. Introduction

Given a set of N targets $V = \{v_1, \dots, v_N\}$, each enclosed by a disk neighbourhood D_i of radius r_i , the Close-Enough Traveling Salesman Problem (CETSP) asks for a minimum-cost closed tour passing through at least one point of every disk. Unlike the classical TSP, only proximity is required: any interior point of D_i constitutes a valid visit. When all radii vanish the problem reduces to the TSP, confirming NP-hardness. This spatial flexibility makes the CETSP a natural model for autonomous routing where exact waypoint precision is neither required nor achievable: UAV mission planning, robotic infrastructure inspection, mobile data collection in wireless sensor networks, and precision agriculture. These are not hypothetical settings: recent work plans unmanned-aerial-vehicle trajectories directly as a CETSP (Cariou et al., 2023), schedules inspection flights over engineering assets (Zhen et al., 2024), and routes vessel-launched UAV swarms across off-shore wind farms (Ismail et al., 2025), in each case exploiting the fact that a target is served from anywhere within sensing range.

Early work addressed this difficulty through Steiner-zone constructions and second-order cone programming (SOCP) reformulations, achieving provable optimality on small instances at exponential runtime cost (Behdani & Smith, 2014; Coutinho et al., 2016); branch-and-bound methods have

since been refined further and now close benchmark instances that were previously open (W. Zhang et al., 2023). Discretisation-based heuristics (Carrabs et al., 2017; Carrabs et al., 2020) extended tractability to medium-sized problems, and a genetic algorithm with SOCP refinement later reported 32 new best-known solutions on standard benchmarks (Di Placido et al., 2022). The problem has also been generalised along several axes (profits attached to targets (Di Placido et al., 2023), multiple vehicles (Carrabs et al., 2025), and explicitly modelled service times at each target (Cui et al., 2026)), which keeps the underlying solvers under continuous pressure. More recently, nature-inspired metaheuristics have been applied to the CETSP, including Harris Hawks Optimisation (Dökeroğlu & Cantürk, 2025) and a parallel fitness-oriented memetic algorithm (Cantürk, 2025). The current state of the art, MA-CETSP, proposed by Lei and Hao (2024), combines a multi-step crossover (MSX) operator with variable-neighbourhood descent (VND) local search inside a diversity-preserving population-management framework, yielding 30 new best-known solutions across 62 benchmark instances and establishing VND-based memetic search as the dominant paradigm for large-scale CETSP; subsequent work has shown that this paradigm scales further still when the population is distributed across cores (Cantürk & Dökeroğlu, 2026).

A second line of development runs alongside this algorithmic one. Machine learning has moved from a peripheral tool to a structural component of modern optimisation, and recent surveys map the resulting design space in detail (Bolufé-Röhler & Tamayo-Vera, 2025; Talbi, 2021): learned components now select operators, tune parameters,

*Corresponding author.

 demir.balemir@tedu.edu.tr (D. Balemir);

deniz.canturk@tedu.edu.tr (D. Cantürk)

ORCID(s): 0009-0005-6589-9533 (D. Balemir); 0000-0002-7606-8701

(D. Cantürk)

control restarts and rank candidate solutions. Within routing specifically, end-to-end neural solvers have matured into a research area of their own (Wu et al., 2026), and the approach has already reached the CETSP (Fan et al., 2025). A parallel and more conservative strand keeps a proven metaheuristic in place and inserts a learned decision at a single point inside it: Q-learning selects among low-level heuristics in a selection hyper-heuristic (Zhao et al., 2023), deep reinforcement learning does the same under uncertainty (Y. Zhang et al., 2022), a Q-learning controller governs the local-search step of a memetic algorithm (W. Zhang et al., 2024), and a learned model predicts which heuristic to run on a given instance (Kärcher & Meyr, 2025). The appeal of this second strand is that the solver's operators and tuning survive intact; what the learned component changes is only how a fixed budget is spent.

The specific form of learned decision that concerns us here is the *surrogate*: a cheap model consulted in place of an expensive evaluation. Surrogate-assisted evolutionary computation is a mature field with its own reviews and taxonomies (He et al., 2023; Jin, 2005), and its scope now extends well past the continuous, low-dimensional problems where it began, into combinatorial and permutation-based search (Liu et al., 2024), including memetic algorithms in which the surrogate decides which individuals are worth refining (Ikeguchi et al., 2025). Two lessons from that literature bear directly on what follows. First, a surrogate need not predict fitness numerically: it is often sufficient, and empirically more robust, to learn a classification or a pairwise relation among candidates and use it to admit or reject them (Hao et al., 2024; J. Li et al., 2022). Second, surrogate use must be managed rather than assumed beneficial, because an inaccurate model biases the search instead of merely slowing it: approximation error acts like noise injected into selection (Zheng et al., 2024), and the accuracy a surrogate must attain before it helps at all has itself become an object of study (Hanawa et al., 2025). This motivates a complementary question: whether the information available before local search supports discrimination useful for filtering. The present paper examines this question through measurements of predictive performance and rank transmission in a memetic CETSP pipeline.

Despite its strong empirical performance, MA-CETSP applies VND to every generated offspring. Because VND dominates the per-iteration runtime, evaluating offspring that will be discarded immediately consumes computation that selective filtering might save. In a preliminary study (Balemir et al., 2026) we augmented MA-CETSP with a survival-analysis-based offspring filter and reported that 13–26% of offspring could be pruned on four benchmark instances while maintaining competitive solution quality.

The scope of that preliminary study limited the strength of its conclusions. Four instances, five seeds and a 500-iteration budget offered limited evidence for separating a filtering effect from run-to-run variation, and no concordance index was reported for any model. The present work extends

the evaluation and finds no statistically detectable benefit at the larger scale. It also examines the predictive and pipeline characteristics that may account for this outcome.

This paper makes four contributions. **(i) A complete surrogate-filtering framework for memetic CETSP.** Nine survival models (Cox PH, ElasticNet Cox, RSF, GBSA, DeepSurv, SSVM, WeibullAFT, MTLR and a k -NN survival estimator) run concurrently on independent islands alongside an in-run no-filter control island, with an adaptive training trigger, validation-calibrated rejection thresholds and a rolling rejection-rate cap. The implementation removes the cross-language inference bottleneck identified as the main limitation of the preliminary study. **(ii) A large-scale evaluation.** On 53 benchmark instances with five seeds per configuration, we detect no statistically significant differences in solution quality or convergence speed relative to the baseline. **(iii) A diagnosis.** We document limited discrimination across the tested model classes and feature families, and identify attenuation of cost ordering through local search, particularly at the LKH stage, as a contributing mechanism. **(iv) A viability gate.** We propose a low-cost, one-sided screening procedure and apply it retrospectively to CETSP. It can support a decision against a filtering design under the assessed conditions; passing it supports further evaluation rather than establishing benefit.

Section 2 surveys related work. Section 3 describes the framework. Section 4 details the experimental setup. Section 5 reports the effect of filtering. Section 6 examines the observed outcome. Section 7 derives the viability gate. Section 8 discusses implications and threats to validity, and Section 9 concludes.

2. Related Work

2.1. Early and Exact Approaches

The CETSP was formally defined by Gulczynski et al. (2006) as a geometric relaxation of the classical TSP in which each target is enclosed by a circular neighbourhood of radius r_i , and any interior point constitutes a valid visit. Initial solution strategies combined supernode construction with Steiner-zone-based local refinement to produce feasible tours. Exact formulations based on Mixed-Integer Programming (MIP) and Second-Order Cone Programming (SOCP) were later developed (Behdani & Smith, 2014; Coutinho et al., 2016), delivering provably optimal solutions for small instances at the cost of exponential runtime growth as problem size increases.

Carrabs et al. addressed this scalability limitation through adaptive discretisation, transforming the continuous CETSP into a sequence of discrete subproblems amenable to classical combinatorial solvers (Carrabs et al., 2017; Carrabs et al., 2020). The accompanying variable-neighbourhood search heuristics achieved tighter bounds and faster runtimes, establishing discretisation-based approaches as competitive alternatives to exact methods.

Exact optimisation of the CETSP has remained active. W. Zhang et al. (2023) present a branch-and-bound algorithm that resolves benchmark instances left open by earlier

exact work, and the problem has been extended in several directions that keep the exact machinery under development: a generalised variant in which targets carry profits (Di Placido et al., 2023), a multiple-vehicle formulation for which upper-bounding procedures were recently proposed (Carrabs et al., 2025), and a version in which the service time spent at each target is modelled explicitly rather than assumed negligible (Cui et al., 2026). Exact methods nonetheless remain confined to instance sizes well below those studied here, which is why heuristic search retains its role.

2.2. Metaheuristic and Hybrid Methods

Population-based metaheuristics have become the dominant paradigm for large-scale CETSP instances. Di Placido et al. (2022) coupled a genetic algorithm with SOCP-based refinement and 2-opt local search, reporting 32 new best-known solutions on standard benchmarks and demonstrating that hybridising evolutionary search with continuous optimisation yields highly competitive outcomes.

The current state of the art is the Memetic Algorithm for CETSP (MA-CETSP) proposed by Lei and Hao (2024), which combines a multi-step crossover (MSX) operator with variable-neighbourhood descent (VND) local search inside a diversity-preserving population-management framework. MA-CETSP produced 30 new best-known solutions across 62 benchmark instances, surpassing all prior methods. Despite its strong performance, the algorithm applies VND to every generated offspring, motivating an assessment of whether selective evaluation could reduce computational cost. Cantürk (2025) subsequently introduced a parallel fitness-oriented memetic algorithm that exploits multi-core architectures to reduce wall-clock time, extended in Cantürk and Dökeroğlu (2026) to substantially larger instances by distributing the population across cores, and Derouiche et al. (2025) proposed deep crossover operators to enrich offspring diversity in genetic search. Working from the geometry rather than the population side, Cariou et al. (2023) combined an evolutionary algorithm with heuristics tailored to the disk neighbourhoods themselves and applied the result to unmanned-aerial-vehicle trajectory planning.

2.3. Learning-Based Optimisation

Machine learning has been integrated into combinatorial optimisation through two principal paradigms, whose design space is catalogued in recent surveys of the field (Bolufé-Röhler & Tamayo-Vera, 2025; Talbi, 2021). End-to-end deep reinforcement learning (DRL) approaches, of which the Attention Model (Kool et al., 2019) is the canonical routing example, and which Wu et al. (2026) survey comprehensively for vehicle routing problems, learn constructive policies for routing problems that achieve near-optimal quality at inference speed. Fan et al. (2025) transferred this paradigm to the CETSP through a Unified Dual-Decoder DRL framework combining a Transformer encoder with k -nearest-neighbour subgraph reasoning. End-to-end learned solvers provide an alternative to inserting learned decisions into an existing search pipeline; the latter retains its established operators and instance-specific tuning.

A complementary strategy retains a high-quality solver and applies learned models selectively to reduce its computational burden. Selection hyper-heuristics learn which low-level heuristic to apply next, using Q-learning (Zhao et al., 2023) or deep reinforcement learning under uncertainty (Y. Zhang et al., 2022); W. Zhang et al. (2024) place a Q-learning controller on the local-search step of a memetic algorithm; and Kärcher and Meyr (2025) predict, per instance, which of several heuristics will perform best. In each case the learned model redistributes a fixed computational budget rather than replacing the search that consumes it. The offspring filter studied in this paper belongs to this family, with one distinction relevant to its assessment: it is asked to predict the outcome of the transformation it is trying to avoid executing.

In our preliminary study (Balemir et al., 2026) we introduced a learning-assisted extension of MA-CETSP in which offspring are scored by a survival analysis model prior to local search, allowing unpromising candidates to be discarded without VND evaluation. On four benchmark instances, 13–26% of offspring were pruned with only marginal reported degradation in solution quality. That study did not, however, report the discriminative power of any of the fitted models, and the present work finds no statistically significant benefit in the larger evaluation of Section 5.

2.4. Surrogate-Assisted Evolutionary Computation

The idea of substituting a cheap learned approximation for an expensive evaluation is long established in evolutionary computation. Jin (2005) surveys fitness approximation in evolutionary algorithms and establishes the design vocabulary still in use: the choice of approximation model, the amount of true evaluation retained, and the burn-in period required before a surrogate is trustworthy (typically 10–20% of the evaluation budget), a guideline we adopt directly in Section 3.6. Jin (2011) reviews subsequent advances and identifies the central open difficulty: a surrogate that is inaccurate in the region the search is currently exploring can actively mislead the population toward false optima, so surrogate use must be managed rather than assumed beneficial. This concern gave rise to the literature on *evolution control* and *model management*, which regulates how often the true objective is consulted; He et al. (2023) review the resulting body of work and the model-management strategies it has produced. Two recent results sharpen the point. Zheng et al. (2024) show that surrogate approximation error is usefully treated as noise injected into selection, rather than as a uniform loss of precision, and Hanawa et al. (2025) study directly how accurate a surrogate must be before it improves search at all, finding that the appropriate model-management strategy itself depends on that accuracy. Both motivate examining whether the available inputs support sufficient predictive accuracy for a particular filtering decision, the question studied here.

Within memetic algorithms specifically, surrogates have been applied not only to approximate fitness but to decide

where local search should be invoked. Lim et al. (2010) generalise surrogate-assisted memetic computation and show that ensembles and locally weighted models can identify individuals that warrant refinement, concentrating an expensive improvement operator on the most promising regions of the search space. The offspring filter studied in this paper belongs squarely to that family: the surrogate does not replace the objective, it gates access to the local search.

Two further developments make that family directly relevant to the CETSP. First, surrogate assistance is no longer confined to continuous decision spaces: Liu et al. (2024) survey surrogate-assisted evolutionary algorithms for expensive *combinatorial* optimisation, and Ikeguchi et al. (2025) build a surrogate-assisted memetic algorithm for permutation-based problems in which a discrete-space surrogate decides where the expensive operator is applied, structurally the closest existing analogue to the filter evaluated here. Second, the surrogate's target need not be a fitness value. Classification-based surrogates learn only whether a candidate is good or poor (J. Li et al., 2022), and relation-based surrogates learn a pairwise comparison between candidates and use it to admit unevaluated solutions into the population (Hao et al., 2024); both are natural fits for an accept/reject gate and need not reconstruct exact fitness values. Alongside these, work on composite and ensemble surrogates (Zhai et al., 2024) and on using explainability to accelerate surrogate-assisted search (B. Li et al., 2024) illustrates how much design effort the field currently invests in improving surrogate quality. Section 6 complements these approaches by examining limitations of the tested inputs and target before further model development is considered.

Two features of the established evaluation practice in this literature motivate the present study. First, surrogates are predominantly assessed by their effect on end-to-end search performance, which conflates the quality of the surrogate with the robustness of the search to a poor one; the standalone discriminative power of the fitted model is reported comparatively rarely. Second, and more consequentially, the transformation that sits *between* the surrogate's inputs and its target is seldom examined. In a memetic algorithm that transformation is the local search itself. Attenuation of the entering cost ordering may limit the usefulness of cost-based pre-local-search descriptors, although it does not exclude predictive information in other representations. To our knowledge this rank-transmission question has not been posed as an explicit diagnostic for surrogate-assisted filtering. Providing such a measure and assessing it alongside predictive performance is the contribution of this paper.

2.5. Survival Analysis Methods

Survival analysis models the time until an event of interest and naturally accommodates censored observations. Cox Proportional Hazards (Cox PH) (Cox, 1972) is the canonical semi-parametric baseline, estimating a log-linear

hazard as a linear function of covariates; an elastic-net-penalised variant (Simon et al., 2011) extends it to correlated or high-dimensional feature spaces. Random Survival Forests (RSF) (Ishwaran et al., 2008) aggregate survival trees grown on bootstrap samples to capture non-linear covariate interactions, while Gradient Boosting Survival Analysis (GBSA) (Hothorn et al., 2006) optimises a Cox partial-likelihood loss stagewise, producing sharper decision boundaries. DeepSurv (Katzman et al., 2018) replaces the linear Cox predictor with a deep neural network to model complex non-linear hazard landscapes, and MTLR (Yu et al., 2011) discretises the time axis and learns a sequence of logistic regressors, implemented here via a neural-network backbone. Survival Support Vector Machines (SSVM) (Van Belle et al., 2007) cast the ranking problem induced by censored data as a structured SVM objective, whereas the Weibull Accelerated Failure Time model (WeibullAFT) (Wei, 1992) directly regresses log-survival time under a parametric distributional assumption. Finally, a k -NN survival estimator assigns risk scores as the negative mean survival time of the k nearest training neighbours (Pedregosa et al., 2011), serving as a non-parametric baseline. Together, these nine models span the spectrum from interpretable parametric methods to flexible deep learners, enabling a systematic comparison of pruning behaviour across diverse modelling assumptions.

The choice to span the spectrum rather than to select a single model is deliberate, and is supported by the current state of the survival-analysis literature. Wiegbe et al. (2024) review deep learning for survival analysis and find a field in which architectural variety has grown much faster than evidence of consistent superiority, and Burk et al. (2026) report a large-scale neutral comparison in which no model class dominates on low-dimensional data: classical Cox-type models remain competitive with tree-based and neural alternatives. We therefore treat the nine models as a sensitivity analysis over modelling assumptions rather than as a ranking exercise, and Section 6.1 interprets their concordance indices in that spirit.

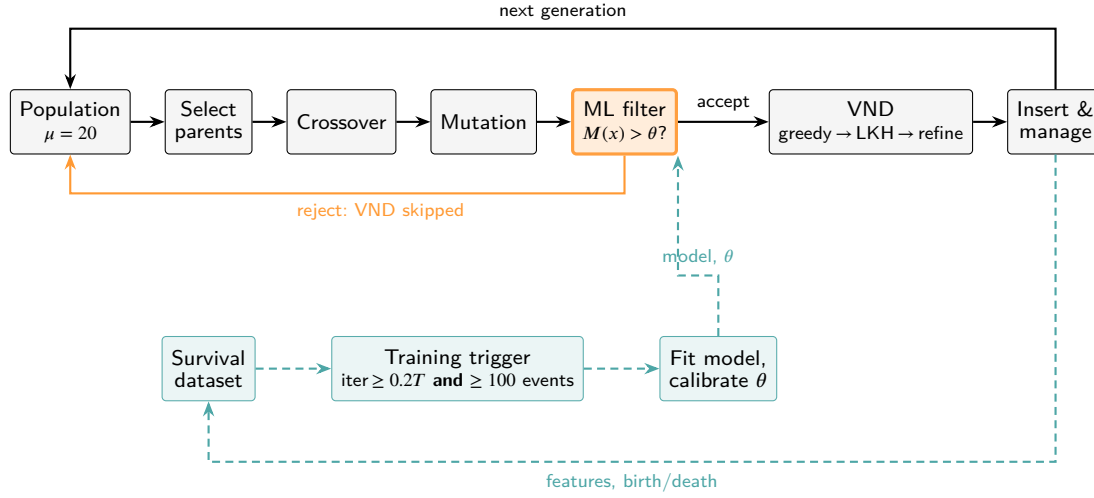
3. Framework

Figure 1 gives the system in one view and Algorithm 1 states it precisely; this section explains each part and the reasoning behind it.

3.1. Baseline: MA-CETSP

Our baseline is the memetic algorithm of Lei and Hao (2024), and we describe it only to the depth the rest of the paper depends upon.

A population P of $\mu = 20$ tours is initialised by k -means clustering of the target disks, followed by an LKH pass and a greedy geometric adjustment on each member. Every iteration draws two parents uniformly, produces one offspring by the k -step crossover operator, applies a mutation with probability proportional to the current stagnation counter, and passes the offspring to variable-neighbourhood descent. Two properties of this loop matter downstream.



Inference: Cox evaluated natively in C++; the other eight served by a resident process over a pipe

Figure 1: The filtered memetic algorithm. The search loop (solid) is the baseline of Section 3.1 with one insertion: the filter sits between mutation and local search, the only position at which a rejection saves the cost it is meant to save. The learning loop (dashed) closes the cycle: the search generates its own training data, a model is fitted once the trigger conditions of Equations (2)–(3) are met, and the calibrated threshold arms the filter for the remainder of the run. A rejected offspring bypasses VND entirely and is neutral for the stagnation counter.

Local search dominates the cost, and it is a pipeline.

The VND stage is not a single operator but a sequence: a cheap greedy repositioning of the visit points, an LKH solve on the induced tour, a second greedy pass, a best-improvement relocate-and-swap over a 50-neighbour candidate list, and a final continuous refinement of the visit points. The LKH solve dominates the runtime, and it is this internal structure that Section 6.4 exploits to localise where information is lost.

Survival is decided by rank, against the current population. After each insertion, every member is assigned

$$f(s) = \alpha \cdot r_{\text{val}}(s) + \beta \cdot r_{\text{div}}(s), \quad \alpha = 1, \beta = 0.96, \quad (1)$$

where r_{val} is the rank of s by tour cost and r_{div} its rank by minimum edit distance to the rest of the population, both scaled to $[0, 100]$. When the population reaches 1.5μ members it is sorted by f and truncated back to μ . Equation (1) is what makes “survival” a well-defined label, and equally what makes it a moving target: both ranks are relative to a population that keeps changing, so an individual’s fate depends on solutions that do not exist when it is evaluated. Section 6.3 returns to this.

An offspring is admitted to the population if it either improves on the incumbent best while differing from every member, or lies at edit distance greater than five from its nearest neighbour in P ; the diversity term in Equation (1) then governs how long it stays.

3.2. Offspring Survival as a Learning Problem

Pre-local-search descriptors are stored on each evaluated offspring. Admitted individuals are written to the log when evicted, yielding a birth iteration b , death iteration d and

duration $t = d - b$. At the ML training trigger, surviving admitted individuals are written as right-censored observations. Unadmitted offspring and terminal survivors on the control island are not written to the historical logs.

Survival analysis accommodates event times and censoring in the training data. However, the offline diagnostics retain positive-lifetime records, excluding immediate evictions as well as unlogged candidates and terminal survivors. The absence of censored records in the control sample therefore reflects selection into the recorded data, not complete observation of all lifetimes. All survival and reconstructed-cohort diagnostics below are conditional on this sample. Assessing predictability across all generated offspring requires complete logging and new experiments.

3.3. Feature Representation

Each individual is described at birth by ten quantities computed from the pre-VND tour: the mean and variance of its edge lengths; the width, height and area of its axis-aligned bounding box; the two coordinates of its centroid; the sum of point-to-centroid distances; the variance of the turning angles at interior vertices; and the pre-VND tour cost. This is the representation inherited from the preliminary study, and it is the one actually deployed.

Two of its properties deserve to be stated here rather than discovered later. Six of the ten (the three bounding-box quantities, the two centroid coordinates and the centroid distance sum) are near-constant within an instance, because every candidate tour visits the same set of disks and therefore occupies almost the same region of the plane; they vary across instances but barely within one, which is the only variation a per-instance model can use. The implemented mean edge length omits the closing edge, so it is closely

related to tour cost but not exactly proportional to it. Section 6.2 quantifies the consequences and tests four alternative representations.

3.4. Survival Models

Nine models are integrated, chosen to span the modelling spectrum rather than to represent a tuned selection. *Cox proportional hazards* provides the interpretable semi-parametric baseline, with an *elastic-net penalised* variant for the collinearity noted above. *Random survival forests* and *gradient boosting survival analysis* capture non-linear interactions by ensembling survival trees, the former by bagging and the latter by stagewise optimisation of the Cox partial likelihood. *DeepSurv* replaces the linear predictor with a multilayer network, and *MTLR* discretises the time axis and learns a sequence of logistic models over it; both use a two-hidden-layer network of 64 units with batch normalisation and dropout. *Survival support vector machines* treat the censored comparisons as a ranking problem, the *Weibull accelerated failure time* model imposes a parametric duration distribution, and a *k-nearest-neighbour* estimator scores an individual by the negated mean survival of its fifteen nearest training neighbours, serving as a non-parametric reference.

Each model exposes the same interface: fitted on the logged features and durations, it returns a scalar risk score in which larger values indicate shorter expected survival. The filter compares that score against a calibrated threshold (Section 3.7) and rejects the offspring when it is exceeded.

3.5. Parallel Architecture and the In-Run Control

A run launches ten independent searches on one machine, one per thread. Nine carry one survival model each; the tenth runs with filtering disabled and serves as a control. Island i is seeded with $\sigma + i$ for a run-level seed σ , and each maintains its own population, log directory, model directory and solver scratch space.

We are deliberate about what this is and is not. There is **no migration** between the islands: no individual, no statistic and no model is exchanged. This is therefore a parallel portfolio with a built-in ablation control, not an island model in the sense the term normally carries in evolutionary computation, and we avoid claiming otherwise. Its purpose is experimental rather than algorithmic (it places all nine treatments and the control in the same run, on the same machine, under the same load), and the reported per-run best is taken over the control alone unless stated otherwise.

The seeding scheme has a consequence we carry into Section 8.4: because island i uses seed $\sigma + i$, a model and the control never share a random stream, so model-versus-control comparisons are unpaired at the seed level and aggregation over instances is what recovers statistical power.

3.6. Adaptive Training Trigger

The preliminary study trained at a fixed iteration, which allocates the same burn-in to an instance that converges in two hundred generations and one that improves for five

thousand. We replace it with a trigger that fires once per run, when both of the following hold:

$$\text{iter} \geq 0.20 \cdot \text{budget}, \quad (2)$$

$$\#\{\text{observed evictions}\} \geq 100. \quad (3)$$

A hard fallback at $0.50 \cdot \text{budget}$ fires regardless of Equation (3), so a population that evicts rarely is never postponed indefinitely.

The two conditions have different rationales. Equation (2) is a burn-in: surrogate-assisted evolutionary computation conventionally withholds the surrogate for the first 10–20% of the evaluation budget so that it is fitted on a population representative of the region being searched (Jin, 2005; Lim et al., 2010). Equation (3) is a sample-size floor. Survival models are constrained by the number of *events*, not observations, and the events-per-variable heuristic asks for roughly ten events per covariate (Peduzzi et al., 1995); with ten covariates this gives one hundred. More detailed sample-size considerations are discussed by Riley et al. (2019). This is an operational training floor, not a guarantee of adequate sample size for every model. The islands share trigger conditions; their realised event counts and training sets may differ.

One design decision here is easy to get wrong and worth stating explicitly. The trigger does *not* consult the stagnation counter, and never resets it. The counter drives both the mutation rate and the stopping rule, and it must advance identically on the control and on every ML island for the comparison to mean anything; an earlier arrangement in which training reset it gave the ML islands a longer search and a lower mutation rate, which confounded exactly the measurement the experiment exists to make.

3.7. Threshold Calibration

A risk score becomes a filter only once a rejection threshold is chosen, and a fixed percentile ignores both instance difficulty and model quality. We select it on a held-out 20% validation split which the model has not been fitted on.

For each candidate percentile $p \in \{50, 55, \dots, 95\}$ we compute the implied rejection rate r and the normalised survival gap

$$g = \frac{\bar{t}_{\text{accepted}} - \bar{t}_{\text{rejected}}}{\bar{t}_{\text{all}}}, \quad (4)$$

which is positive when accepted individuals do outlive rejected ones, and maximise

$$J(r, g) = g \cdot \min\left(1, \frac{r}{r^*}\right) - \lambda \max(0, r - r^*), \quad (5)$$

with a target rejection rate $r^* = 0.20$ and a penalty weight $\lambda = 2$.

The shape of Equation (5) is a correction to the obvious choice. Maximising $r \cdot g$ appears to balance pruning against discrimination, but on a validation split the rejection rate is mechanically $(100 - p)/100$, so that objective contains a built-in pull toward low percentiles that has nothing to do

with the model. The ramp term instead grants full credit for discrimination once a target rejection rate is reached, and the penalty term discourages exceeding it. Percentile candidates are not restricted, only the resulting rate is softly capped, so models whose risk distributions require a low percentile retain access to one.

3.8. Rolling Rejection-Rate Cap

A threshold calibrated on the training distribution can become miscalibrated later in the run, since the offspring generated after the model is fitted are drawn from a population the model never saw. Left unchecked this produced rejection rates above 40% on some seeds, collapsing diversity and stalling the search.

We therefore add a runtime guard. Every 50 filter-eligible offspring the realised rejection rate over the window is measured; if it exceeds 30%, the filter is suspended for the following window and the check repeats. The guard is a safety mechanism rather than a tuning device, and Section 5 reports rejection rates well below the cap.

3.9. Implementation

The preliminary study named cross-language inference latency as the obstacle to converting pruning into wall-clock savings: each prediction opened a fresh Python process, and the overhead was comparable to the local-search call it was meant to avoid. Two changes remove it.

Eight of the nine models are served by a *resident* scoring process started once per island and held open for the duration of the run, communicating over an inherited pipe pair: the solver writes one JSON line and reads one score back, so a prediction costs a round trip rather than an interpreter start-up. The process is restarted only when the model is refitted, and is respawned automatically if it dies. The Cox model bypasses the boundary entirely: its coefficients and standardisation statistics are read from disk and the linear predictor is evaluated natively, so Cox islands invoke no external process at all.

Algorithm 1 states the resulting loop. Two details in it are easy to overlook. The filter is placed after mutation and before local search, which is the only position at which a rejection saves the cost it is meant to save. And a rejected iteration is treated as *neutral* for stagnation: it neither resets the counter nor advances it. An offspring that never reached local search is not evidence that the search has converged, and counting it as such would let an aggressive filter trigger a premature stop and then report the resulting short run as a speed-up.

Algorithm 1 One island of the filtered memetic algorithm

Require: instance I , budget T , patience threshold ρ , model M

```

1:  $P \leftarrow \text{INITIALISE}(I)$ ;  $s^* \leftarrow \text{BEST}(P)$ 
2:  $\pi \leftarrow 0$ ; trained  $\leftarrow$  false
3: for iter = 1 ...  $T$  do
4:    $s_a, s_b \leftarrow \text{SELECTPARENTS}(P)$ 
5:    $s \leftarrow \text{CROSSOVER}(s_a, s_b)$ 
6:   if  $\text{RAND}(1000) < \pi$  then  $s \leftarrow \text{MUTATE}(s)$ 
7:   rejected  $\leftarrow$  false
8:   if trained and not suspended then
9:      $x \leftarrow \text{FEATURES}(s)$   $\triangleright$  pre-VND, no local search
    yet
10:    if  $M(x) > \theta$  then
11:      rejected  $\leftarrow$  true  $\triangleright$  skip VND entirely
12:    end if
13:  end if
14:  if not rejected then
15:     $s \leftarrow \text{VND}(s)$   $\triangleright$  dominant cost
16:     $P \leftarrow \text{INSERT}(P, s)$ ;  $\text{MANAGE}(P)$   $\triangleright$  Eq. (1)
17:  end if
18:  if  $\text{BEST}(P) < s^*$  then
19:     $s^* \leftarrow \text{BEST}(P)$ ;  $\pi \leftarrow 0$ 
20:  else if not rejected then
21:     $\pi \leftarrow \pi + 1$   $\triangleright$  rejections are neutral
22:  end if
23:  if trigger fires (Eqs. (2)–(3)) then fit  $M$ , calibrate  $\theta$ ,
    trained  $\leftarrow$  true
24:  if  $\pi \geq \rho$  then break
25: end for
26: return  $s^*$ 
```

4. Experimental Setup

4.1. Benchmark Instances

We use the 68 CETSP instances established by prior work: overlap-ratio families derived from TSPLIB problems, arbitrary-radius variants, structured synthetic families and industrial car-door welding instances, with up to one thousand targets. A 15-instance development set informed the framework design. The remaining 53 instances form the end-to-end test set reported in Section 5 (Table 1).

The retrospective diagnostics in Sections 6–7 use separate subsets of available logs. The five-instance model and oracle subset includes one development instance (bonus1000); the 25-instance VND calibration includes eight development instances. These are exploratory diagnostics, not an independent test of the gate. The released instance manifest records every membership. The instrumented stage analyses use island-0 records; model, oracle, lineage and full-VND calibration use island-9 records.

4.2. Protocol and Parameters

Every configuration uses five base seeds: 1, 11, 21, 31 and 41. Each run has ten islands and an iteration budget of $T = 5000$, giving 265 runs of ten islands each, or 2650 island-runs in total. Search parameters follow the baseline:

Table 1

Test-set composition. Groups follow the standard naming: or2, or10 and or30 denote overlap ratios of 0.02, 0.1 and 0.3; rdmRad denotes arbitrary radii; real_world are the car-door instances. “Filter active” counts instances on which at least one ML island reached the training trigger.

Group	Instances	Filter active
varied	22	19
rdmRad	11	8
or2	6	6
or30	6	6
or10	5	5
real_world	3	3
Total	53	47

population $\mu = 20$, diversity coefficient $\beta = 0.96$, insertion distance threshold 5, candidate neighbourhood size 50, k -means initialisation, k -step crossover, and edit distance as the population distance measure.

Early stopping fires after $\rho = \max(T/10, 50) = 500$ consecutive non-improving iterations. The rule is identical on the control and on all nine ML islands, and, as described in Section 3.6, is never touched by the training trigger. Consequently the budget is adaptive in practice: instances that converge quickly stop early, and the median run terminates near iteration 1400 rather than exhausting the 5000 available. Filter-specific constants are those given in Section 3: burn-in $0.20T$, event floor 100, hard fallback $0.50T$, target rejection rate 0.20, and a rolling cap of 30% over windows of 50 filter-eligible offspring.

All experiments were run on a single machine, specified in Table 2. Two details of it bear on the results. The ten islands of a run execute concurrently as threads of one process, which fits within the 14 physical cores, but they still contend for memory bandwidth and for the LKH subprocesses each of them spawns; this is the concrete basis for the timing caveat in Section 4.3. And although the machine has a discrete GPU, the installed PyTorch build is CPU-only and CUDA was unavailable, so DeepSurv and MTLR were fitted and evaluated on the CPU like every other model. No model in this study received hardware acceleration that the others did not.

CPU execution matches the evaluated workload: ten independent searches, a dominant LKH stage, and models fitted once per run on roughly one thousand rows of ten features. Inference scores one offspring at a time before the next search step proceeds, making per-request latency relevant (Section 3.9). GPU execution was not benchmarked, so the reported timing comparisons apply to the CPU configuration.

4.3. Evaluation Measures

For each instance we report mean best cost over five runs, rejection counts and total wall-clock runtime including training. Table 3 averages run-level ratios: degradation is $100(f_{mi}/f_{0i} - 1)$ and speedup is t_{0i}/t_{mi} , matched by instance

Table 2

Execution environment. All reported runs were produced on this configuration.

CPU	Intel Core i7-12700H 14C / 20T, 2.7 GHz L2 11.5 MB, L3 24 MB
Memory	32 GB DDR4-3200
Storage	NVMe SSD
GPU	unused (CPU-only build)
OS	Windows 11 Pro (26200)
Solver	C++17, MSVC 2022, /O2
Build	CMake 4.2
LKH	2.0.11
Python	3.10.9 scikit-survival 0.25.0 lifelines 0.30.0 scikit-learn 1.7.2 PyTorch 2.11 (CPU) pycox 0.3.0

and run identifier. The matches have distinct random seeds because island i uses $\sigma + i$. The ratio of mean costs is generally different from the mean of these ratios.

Two-sided Wilcoxon signed-rank tests (normal approximation, no continuity correction) use model-minus-control per-instance mean differences, assuming symmetry of the difference distribution. Quality tests use all 53 instances; runtime tests use the 47 instances with at least one recorded rejection. Differences are rounded to ten decimal places before testing to avoid floating-point rank artefacts.

Ten islands share a machine, so runtimes include resource contention. Rejected iterations skip VND and are neutral for the stagnation counter. Consequently, iterations do not represent a fixed computational budget across treatments. Runtime to the stopping rule is not time to a common target quality. No compute-weighted speedup is reported: per-offspring VND timings were not logged.

5. Effect of Survival-Based Filtering

5.1. Solution Quality

Table 3 reports the outcome. Mean best cost differs from the control by between -0.009% and $+0.049\%$; the entire spread across nine models is under six hundredths of one percent. No model is significantly different from the control under a Wilcoxon signed-rank test over per-instance means (Table 4), the smallest p -value being 0.31.

Figure 2a shows the same result at instance resolution. Per-instance differences include both signs, with medians near zero. The aggregate therefore obscures variation across instances; the plot alone does not establish symmetry of the difference distribution.

These results indicate no statistically detectable effect in *either* direction under the evaluated protocol; they do not establish equivalence. Two models show a nominally

Table 3

Effect of each survival model relative to the in-run no-filter control, over 53 test instances $\times$ 5 seeds. Degradation averages matched run-level cost ratios; positive means worse. No difference is statistically significant ($p \geq 0.31$ for quality, $p \geq 0.15$ for speed).

Model	Mean best	Degr. (%)	Speedup	Reject (%)
Baseline (no ML)	905.956	+0.000	1.000	0.00
RSF	906.253	−0.009	1.008	2.01
KNN	905.867	−0.003	1.040	5.03
ElasticNet	906.214	+0.001	1.055	5.38
WeibullAFT	906.025	+0.002	1.051	4.85
GBSA	906.026	+0.009	1.019	4.70
DeepSurv	906.048	+0.011	1.030	4.21
MTLR	905.969	+0.014	1.033	4.42
SSVM	906.248	+0.018	1.084	6.51
Cox PH	906.415	+0.049	1.041	8.24

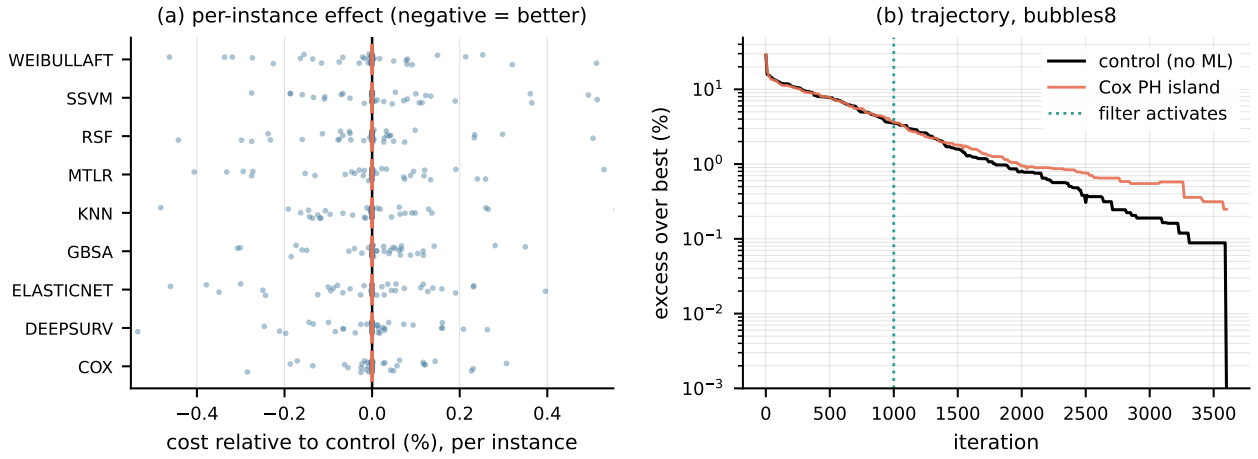


Figure 2: (a) Per-instance effect of every model relative to the control over the 53 test instances. Each point is one instance and the tick marks the median; this is the quantity the signed-rank test operates on, and it is centred on zero for every model. (b) A representative trajectory on an instance whose runs continue well past the activation point, so that the filter is actually operating over the plotted window. A single ML island is shown rather than the mean of all nine: this compares one treatment with the control without pooling distinct models. The curve is descriptive and is not a time-to-target comparison.

negative degradation, but the data provide insufficient evidence to interpret these differences as improvements. The differences are far smaller than the between-seed variation, none approaches significance, and the per-group breakdown shows the sign of the effect flipping between groups for the same model: DeepSurv, for instance, records -0.226% on the *or10* group and $+0.096\%$ on *varied*. Sign changes of this kind are consistent with run-to-run variability. Section 6 examines potential explanations for the outcome.

5.2. Convergence Speed

Speedups relative to the control range from 1.008 to 1.084, and none is significant ($p \geq 0.15$). Two caveats limit their interpretation. First, the ten islands execute concurrently on one machine, so per-island wall-clock times are measured under contention and a model whose neighbours finish early is measured under lighter load than one whose neighbours persist. Second, and more fundamentally, an iteration is not a constant unit of work once filtering is enabled: a

rejected offspring consumes an iteration without a VND call, so an ML island performs fewer local searches per iteration than the control by construction. Any comparison expressed per iteration therefore uses a different computational budget. The speedups here describe total runtime to the stopping rule, not time to a common target quality.

Figure 2 b shows a trajectory directly, with the activation point marked; its visual proximity is descriptive, without a separate trajectory-level hypothesis test.

Rejection rate and speedup are strongly rank-correlated across the nine models (Spearman $\rho = 0.817$, $p = 0.007$). Skipping VND reduces the number of local-search calls by construction; the cross-model association does not isolate its causal effect on wall-clock runtime. Establishing an end-to-end benefit additionally requires considering quality and total computational cost.

Table 4

Wilcoxon signed-rank tests of each model against the control, paired by instance. n is the number of instances contributing a non-zero difference. No test is significant at the 0.05 level, for quality or for speed.

Model	Quality		Speed	
	n	p	n	p
KNN	38	0.313	47	0.949
GBSA	41	0.361	47	0.208
SSVM	40	0.464	47	0.236
WeibullAFT	38	0.557	47	0.657
Cox PH	39	0.567	47	0.857
DeepSurv	40	0.572	47	0.290
ElasticNet	39	0.640	47	0.841
MTLR	36	0.888	47	0.498
RSF	39	0.944	47	0.150

5.3. Filter Activity

To interpret the absence of a statistically detectable effect, we quantify how much filtering occurred.

Where training occurred, the trigger fired at iteration 1000 (the budget floor of 20% of the 5000-iteration budget), meaning the event-count condition was already satisfied in those runs. Because the search terminates on stagnation rather than exhausting the budget, however, not every run reaches that point. Of the 2385 ML island-runs, 852 (35.7%) stopped before iteration 1000 and therefore execute without active filtering; at the instance level, 6 of the 53 instances have no recorded rejections. The released summaries include both the full set and the 47-instance active subset.

The remaining 1533 runs reached the trigger iteration; three stopped exactly there with no subsequent filtering window. The median filtering window is 1083 iterations, or 52% of the run's length, and the mean number of offspring rejected per trigger-reaching run ranges from 73 for RSF to 375 for Cox (medians 41.5 and 211.5). Aggregated over the experiment the filter rejected between 12,653 and 65,265 offspring depending on the model, each rejection eliminating one local-search call.

The filter was active in more than fifteen hundred runs, with a median active window exceeding half the run length, and removed tens of thousands of offspring from local search. No statistically significant change in quality or speed was detected. These measurements indicate that limited activation alone is insufficient to account for the aggregate outcome.

5.4. Threshold Behaviour

Two properties of the calibrated thresholds anticipate the diagnosis.

The first is convergence. Although the percentile search ranges over 50–95 and is run independently for every model, every instance and every seed, the selected percentile clusters tightly at 82.9–85.8 across all nine models. This concentration across heterogeneous model families suggests that the

shape of the calibration objective contributes substantially to threshold selection.

The second is more direct. The calibration objective rewards a positive survival gap: accepted offspring should outlive rejected ones on the held-out split. Between 12.6% (Cox) and 24.1% (MTLR) of calibrations return a *negative* penalised objective. Equation (5) subtracts an over-rejection penalty, so a negative objective does not necessarily imply a negative survival gap. These frequencies describe calibration outcomes, while Section 6 measures discrimination directly. The gap uses observed durations, including censored durations on ML islands, so it is a calibration heuristic rather than a censoring-adjusted survival estimate.

6. Diagnosis

Section 5 found no statistically detectable effect. We examine the tested models, representations and target, then cost-rank transmission. Model, oracle and lineage comparisons use run-aware cross-validation. Univariate features and partial-VND scores are direct concordance measurements. The legacy population-feature multivariate analysis uses shuffled solution-level folds and is exploratory. All diagnostics retain the sample-selection limitations of Section 3.2.

6.1. Discrimination Across Model Classes

We first examine whether greater model flexibility improves discrimination on the deployed feature set.

Establishing this required a measurement the online pipeline does not provide. Of the nine deployed models only Cox records a concordance index during training; the other eight persist a rejection threshold and nothing else, so their discriminative power is invisible at run time. We therefore refitted all nine offline on control-island logs. Diagnostic adjustments use a 0.01 penalty for Cox and Weibull AFT, and 100 rather than 200 final-fit epochs for neural models. The AFT diagnostic uses negative expected survival rather than negative median survival. We evaluated these model families with run-aware cross-validation on the five diagnostic instances (Table 5).

The nine models span linear, penalised linear, parametric, kernel-based, non-parametric, tree-ensemble and deep-network hypothesis classes, and every one of them lands between 0.495 and 0.544 (Figure 3). Two additional measurements on different samples of the same log collection give nearby values: a three-model diagnostic returns 0.532, 0.530 and 0.535 for Cox, RSF and GBSA, and the PRE_GEOM row of Table 6 agrees to within 0.013.

The results provide no evidence of a consistent gain from greater model flexibility under the tested settings. They motivate examining the inputs and prediction target alongside model choice. They do not establish the best achievable performance of other architectures or tuning strategies.

We stress that this is not a ranking of the nine models, and the differences within the table should not be read as one. A spread of 0.049 concordance across nine model families, all within 0.045 of chance, is not a basis for recommending any of them; Burk et al. (2026) report in a large-scale

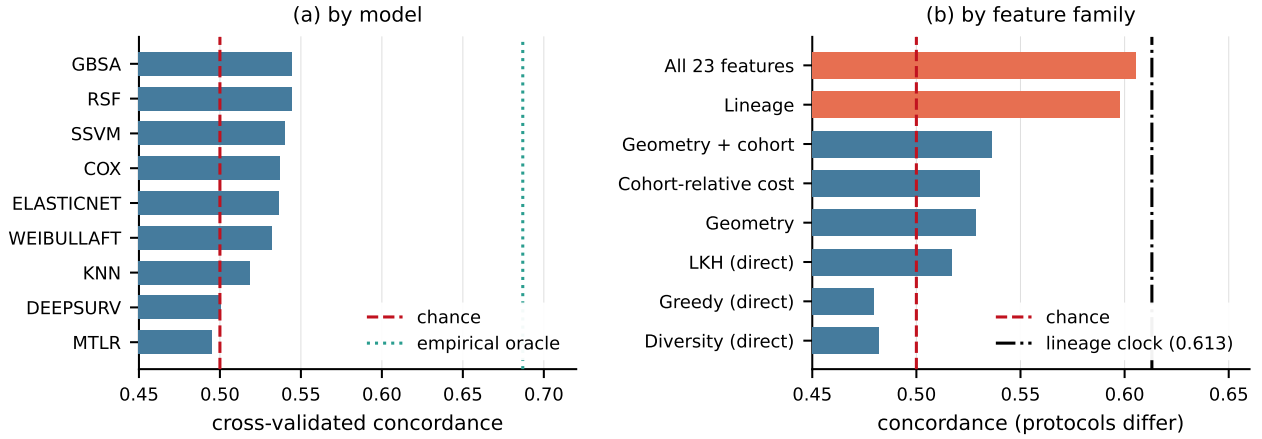


Figure 3: The diagnostic landscape. (a) Nine model families under the diagnostic settings against the chance level and against the empirical oracle reference of Section 6.3; the spread across seven hypothesis classes is smaller than the distance from any of them to the oracle. (b) Feature families with mixed protocols: diversity and partial-stage bars are direct scores; the other bars use run-aware models. The dashed clock reference applies to the lineage sample. The two highlighted bars are measured on the shorter, drift-heavier runs described in Section 6.2; the relevant comparison is with the index baseline measured on the same runs.

Table 5

Cross-validated concordance of nine survival model families under the diagnostic settings, using the deployed feature set and averaged over five instances spanning the structural groups ($n = 17,568$ logged solutions). Chance is 0.500. The tested models show limited discrimination; the values are not a performance ranking.

Model	Class	C
RSF	tree ensemble	0.544
GBSA	gradient boosting	0.544
SSVM	kernel / ranking	0.540
Cox PH	linear	0.537
ElasticNet	penalised linear	0.536
WeibullAFT	parametric AFT	0.532
KNN	non-parametric	0.518
DeepSurv	deep network	0.501
MTLR	deep network	0.495

neutral comparison that even on genuine survival data with real signal, model classes separate far less cleanly than the methods literature implies. What the table supports is limited discrimination under the evaluated representations and training protocol.

These scores describe the positive-lifetime eviction sample. Zero-lifetime records and unlogged terminal survivors are absent. Their exclusion may change the apparent discrimination; a low recorded censoring rate does not remove this selection concern.

6.2. Discrimination Across Feature Families

We next examine sensitivity to the representation. Because features can be prototyped offline against the logged solutions without modifying the solver or repeating any experiment, we were able to test four representation families. These exploratory analyses use different samples and validation protocols, as stated above.

Deployed geometry. Taken individually, none of the ten features in production shows substantial discrimination: every univariate concordance lies within 0.019 of chance, and the two centroid coordinates within 0.005. The signs are not even consistent across instances. Inspection explains why. Six of the ten (the three bounding-box quantities, the two centroid coordinates and the centroid distance sum) are effectively constant within an instance, because every tour visits the same disks and therefore spans nearly the same region; and mean edge length is related to pre-VND tour cost, although its implementation omits the closing edge.

Population-relative cost. The eviction rule ranks an individual against the current population, so scale-free relative quantities are a more principled choice than absolute geometry: the percentile of an offspring's pre-VND cost within the cohort alive at its birth, its ratio to the cohort median and minimum, its z-score, and its ratio to the running best. Multivariate concordance over this family reaches 0.529–0.542.

Diversity. Survival is governed by value rank *and* distance rank, so we examined a proxy: directed Chamfer distance between pre-VND point sets, using at most 80 points per tour and 15 recorded cohort members. Mean and minimum distances have direct concordances 0.478 and 0.482. These raw feature scores may have a protective direction below 0.5; they do not directly test the solver's edit-distance criterion.

Partial local search. Finally we tested whether a cheap prefix of the pipeline provides survival discrimination: we scored recorded costs after the greedy stage and after LKH. No early-exit policy was deployed in this probe. The corresponding concordances are 0.479 and 0.517; the completed VND cost reaches 0.651, by which point the expense the filter exists to avoid has already been incurred.

Lineage. The four families above all describe the offspring itself, and so must survive the local search to be

Table 6

Empirical oracle reference. Concordance averaged after taking the best of Cox, RSF and GBSA within each instance (run-aware folds; optimistic model selection) over five instances spanning the structural groups. ORACLE receives recorded post-VND information, unavailable to a filter that skips VND. Its fitted performance is a diagnostic reference rather than a formal upper bound.

Feature set	Information used	C
PRE_GEOM	deployed geometry	0.529
PRE_POP	cohort-relative cost	0.530
PRE_ALL	best legitimate set	0.536
DRIFT	birth iteration only	0.536
ORACLE	post-VND outcome	0.687
ORACLE_ALL	oracle + all pre-VND	0.688

useful. The remaining family describes where it *came from*, and does not pass through the local search at all: the cost and fitness of both parents, which are population members that have already been evaluated; the two offspring-to-parent edit distances measured before mutation; and whether mutation fired. “Offspring of two strong, mutually distant parents” is a prior on offspring quality that no geometric descriptor can express. This family could not be prototyped from the existing logs, since parent identity was not recorded, so we instrumented the solver and re-ran the affected instances to obtain it.

Lineage features score 0.598 on their own and 0.605 combined with everything else. On the same data the birth iteration *alone* scores 0.613. Twenty-three combined features (geometry, relative cost and lineage), fitted by three model classes, do not beat a single integer recording when the offspring was created. (Absolute levels are elevated here relative to Table 5 because these runs are short and therefore drift-heavy; the comparison against the drift control, not the level, is what carries the conclusion.)

This comparison illustrates the role of the drift control: an apparent increase from 0.53 to 0.60 across different samples does not by itself establish more useful predictive information. We recommend a time-index baseline for feature evaluations on improving search trajectories and return to this in Section 7.

The tested representations show limited discrimination on their recorded samples. The oracle and lineage experiments include matched drift controls; there is no matched clock comparison for every exploratory family. These findings motivate examining the target and recorded post-step information (Figure 3 b).

6.3. An Empirical Oracle Reference

To examine the target, a model is given recorded *post*-VND information (the cost VND actually produced, that cost’s rank within the cohort alive at the offspring’s birth, and the relative gain VND delivered), none of which a real filter can observe, since obtaining them requires running the very local search the filter exists to skip. This oracle provides

an empirical reference for survival prediction with additional information. A fitted model on these descriptors does not establish a universal upper bound for all pre-VND predictors.

Table 6 gives the result, and it supports three conclusions.

Recorded post-VND information improves discrimination. The oracle reaches $C = 0.687$, above the tested pre-VND representations but below perfect prediction. Survival depends on subsequent population changes: an offspring is evicted as better or more diverse individuals accumulate around it. These future events are not directly observed at birth, although their effects may be partly predictable. The result motivates considering target uncertainty alongside representation and model limitations.

Pre-VND features add little in this evaluation. Augmenting the oracle with every pre-VND feature we engineered moves the score from 0.687 to 0.688. This small observed increment suggests limited additional predictive value under the tested protocol; it does not establish conditional independence.

The pre-VND features match the drift reference. A relevant result in the table is the equality of PRE_ALL (0.536) and DRIFT (0.536). The best legitimate feature set we could construct (ten geometric descriptors plus four population-relative cost statistics, over five instances) matches the birth-iteration baseline. This suggests that part of the apparent predictive performance reflects search progress. Such pooled discrimination does not establish an ability to distinguish contemporaneous candidates, which is the relevant comparison for filtering.

One limitation should be stated plainly. Survival is governed by $fitness = value_rank + \beta \cdot distance_rank$. Neither population rank is exactly reconstructible. Unlogged or unsampled live members are absent, and the cost statistics describe the recorded cohort. For distance rank, edit distance is computed after VND against the current population, and only pre-VND coordinates are recorded. The oracle thus uses an incomplete description of the post-VND state. Its measured performance may understate what a richer model and fuller information could achieve, which limits the conclusions that can be drawn from this check alone.

6.4. Cost-Rank Transmission Through Local Search

The oracle analysis of Section 6.3 addresses prediction of the label. To examine the transformation of pre-VND cost information, we instrumented the local search to record the tour cost after each of its internal stages, and tracked how the ordering of offspring propagates through the pipeline (Figure 4 and Table 7).

Rank transmission varies by stage and instance (Figure 4). Greedy-stage correlations span 0.555–0.990; the two larger values show substantial preservation, while the third is already attenuated. LKH-stage correlations span 0.516–0.646, and the closing stages span 0.416–0.577. This identifies several potential sources of rank attenuation rather than a single universal loss point.

Spearman correlation of the offspring cost ordering, stage to stage

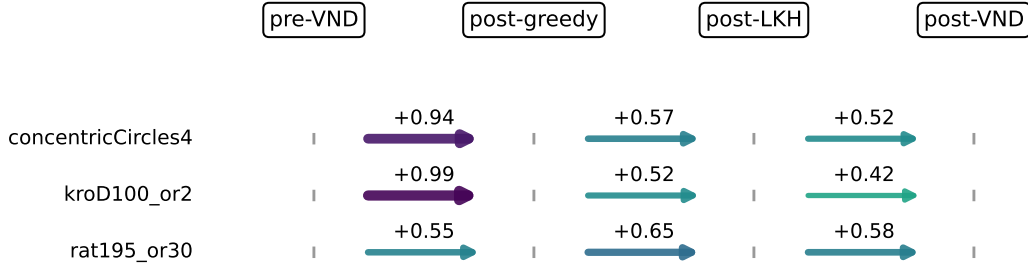


Figure 4: Propagation of the offspring cost ordering through the local-search pipeline. Arrow thickness and colour encode the Spearman correlation between the ordering entering and leaving each stage. Transmission varies by stage and instance; substantial greedy-stage preservation occurs on two of the three.

Table 7

End-to-end correlation between pre- and post-local-search cost ordering in recorded positive-lifetime solutions. The cohort column correlates percentiles within reconstructed recorded cohorts; it does not recover the full live population or restrict every scored pair to the same birth time. Stage-by-stage values are given on Figure 4.

Instance	pooled	cohort percentile
concentricCircles4	+0.48	+0.15
kroD100_or2	+0.13	+0.08
rat195_or30	+0.48	+0.21

Pooled pre/post correlations span 0.129–0.479; correlations of recorded-cohort percentiles span 0.080–0.205. The latter adjusts for relative standing at birth but still pools observations over time and uses incomplete cohorts. It is a temporal diagnostic, not proof that all search-phase drift has been removed.

6.5. Rank–Gain Association Within Local Search

Figure 5 examines a mechanism consistent with this attenuation. For each offspring we measured its cost rank within its recorded cohort at birth, and the relative improvement LKH then delivered. The two are positively rank-correlated: the contraction coefficient κ_{LKH} (the Spearman correlation between those two quantities, computed within each run and averaged across runs) is 0.921, 0.944 and 0.820 on the three instances. Lower-ranked inputs tend to receive larger relative LKH improvements.

This pattern is consistent with compensatory improvement (Figure 5): lower-ranked inputs tend to receive larger relative gains. Together with the stage-wise correlations, it suggests a mechanism for attenuation of entering cost order. Rank–gain association alone does not measure a reduction in the spread of costs.

These measurements are consistent with the limited discrimination observed across the tested models and pre-VND feature families, the small increment from adding those features to the oracle, and the contribution of temporal drift to

pooled correlations. They do not establish that all predictive information in the pre-VND state is absent. In particular, a rank–gain correlation alone does not determine post-step rank preservation or the usefulness of other descriptors.

The practical implication is to assess rank transmission and predictive value before investing further in a filtering design. In the evaluated system, their joint evidence supports reconsidering pre-VND survival filtering. This motivates the one-sided screening procedure in the next section.

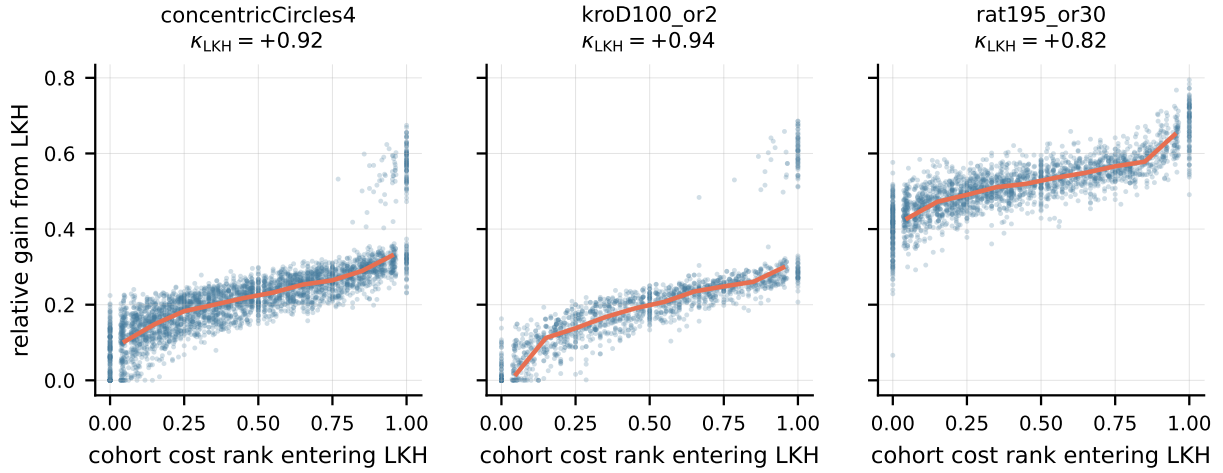


Figure 5: LKH rank–gain association. Each point is one recorded offspring: its post-greedy cost percentile within its recorded birth cohort, against the relative improvement LKH returned. The line is the binned median. Its upward trend indicates larger gains for lower-ranked inputs; rank preservation also depends on the gain magnitudes and their variation. Each panel is titled with the contraction coefficient κ_{LKH} , the Spearman correlation between the two plotted quantities, computed within each run and averaged across runs.

7. A Viability Gate for Pre-Local-Search Filtering

The diagnosis of Section 6 is specific to our system, but the way it was reached is not. Each step measured a property of the pipeline rather than of the surrogate, and every one of those measurements could have been taken *before* the surrogate existed. This section states them as a reusable screening procedure, calibrates the central one, and applies the result retrospectively to our own project.

7.1. Three Checks

The procedure comprises three measurements, each addressing a potential constraint on pre-local-search filtering. They use instrumented logs and offline fits, with enough independent runs for run-aware validation.

Check 1: Is the label reachable? Fit a model on information that is available only *after* the expensive step, and measure how well it predicts the intended target. This is the empirical oracle of Section 6.3. Limited discrimination even with post-step information motivates reassessing the target, representation and available data. Its interpretation depends on the completeness of that information and the fitted models; it is not a formal upper bound for all pre-step predictors.

Check 2: Does the pipeline preserve the ordering? Instrument the expensive step to record its input and output cost, and compute the *contraction coefficient*

$$\kappa = \rho_s(r_{\text{in}}, g), \quad (6)$$

where ρ_s is Spearman’s rank correlation, r_{in} is a candidate’s cost percentile within the reconstructed recorded cohort at its birth, and g is the relative improvement of that same step. We distinguish κ_{LKH} (post-greedy to post-LKH) from κ_{VND} (pre-VND to final VND). As $\kappa \rightarrow 1$, lower-ranked

inputs tend to receive larger relative gains. Whether these gains substantially alter the output ordering also depends on their magnitudes and the input costs. Interpret κ alongside direct input–output rank correlations and a calibration for the pipeline under study.

Check 3: Does the feature set beat the clock? Evaluate any candidate representation against a baseline consisting of the iteration index alone. Data generated by an improving search carries a strong temporal trend, and any feature correlated with time may appear predictive in a pooled evaluation without distinguishing contemporaneous candidates. A gain over the clock is a reason for further within-cohort and deployment evaluation, rather than proof of benefit. The lineage family of Section 6.2 scored 0.598, while its matched iteration-index baseline scored 0.613. Comparisons across different samples are exploratory.

7.2. Calibrating the Coefficient

We examine full-VND rank–gain dependence on 25 diagnostic instances. For each, we use up to three recent runs and the last 800 records in numeric file order, retaining positive-lifetime records and at least three recorded cohort peers. Selected runs, counts and input hashes are released. Cohorts are incomplete; original membership is held fixed throughout this sensitivity experiment.

For each of eleven nominal dependence settings, a Gaussian copula reassigns the observed gains among candidates with valid pre-step cohort percentiles. Five reassignments are drawn per run using seed zero; other gains stay fixed. The gain multiset is preserved, while $q'_i = p_i(1 - g'_i)$. Ties and finite samples mean the realised κ_{VND} need not equal its nominal setting, so both are reported. This does not simulate new survival trajectories or filtering.

Cost-rank concordance C_{rank} is computed directly over pairs with distinct output percentiles: agreeing input order

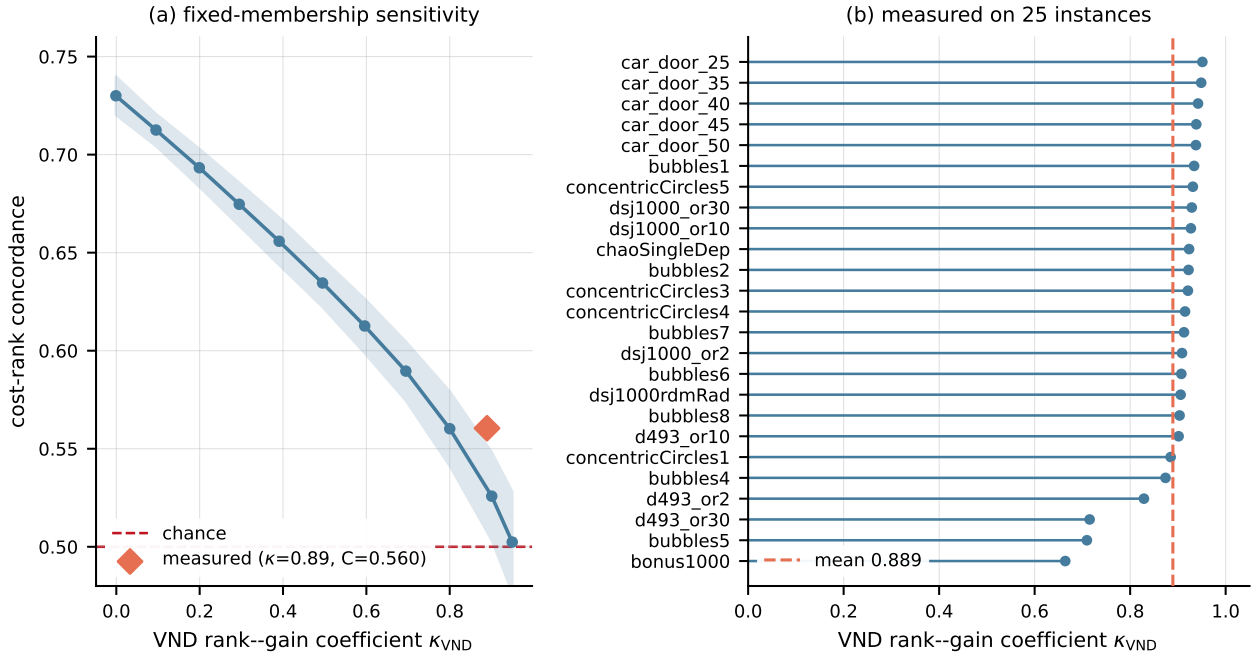


Figure 6: (a) Cost-rank concordance versus mean realised full-VND rank-gain coefficient. The band is one SD across instances; the diamond marks observed data. (b) Observed coefficients across the 25 diagnostic instances, including eight development instances. This is a conditional sensitivity analysis, not a universal viability threshold.

Table 8

Full-VND sensitivity analysis on recorded cohorts. The first column is nominal dependence; the second is mean realised κ_{VND} . The final columns report mean cost-rank concordance and between-instance SD.

Nominal	Realised	C_{rank}	SD
0.00	-0.001	0.730	0.010
0.10	0.095	0.713	0.008
0.20	0.199	0.693	0.010
0.30	0.295	0.675	0.011
0.40	0.391	0.656	0.012
0.50	0.495	0.635	0.012
0.60	0.596	0.613	0.014
0.70	0.695	0.590	0.015
0.80	0.800	0.560	0.019
0.90	0.901	0.526	0.022
0.95	0.950	0.502	0.026

receives one, tied input order one half, and reversed input order zero. Output ties are excluded. This score differs from rescaled Kendall τ_b and from survival concordance. Results are averaged over repeats and runs, then equally across instances.

Table 8 and Figure 6 show decreasing rank concordance as dependence increases under the specified reassignments. The observed mean is $\kappa_{\text{VND}} = 0.889$, with $C_{\text{rank}} = 0.560$. At the nominal zero setting, mean concordance is 0.730; at the largest setting it is 0.502. Zero rank-gain correlation does not imply perfect order preservation, since individual gains still vary.

Table 9

The gate applied to the system of Section 3. Check 1 is the empirical oracle of Section 6.3; check 2 is Equation (6) for full VND, averaged over 25 instances; check 3 compares the best tested pre-VND feature set against the iteration index alone. The checks are interpreted jointly for this design, without universal pass/fail cut-points.

Check	Value
1. Oracle concordance	0.687
2. VND κ	0.889
3. Features / clock	0.605 / 0.613

The observed point need not lie on the synthetic curve: the copula changes dependence beyond a single correlation, and the fixed recorded membership omits feedback through survival. We therefore make no calibrated-performance guarantee. The oracle's survival concordance of 0.687 is a separate diagnostic of a different target and cannot validate this cost-rank curve.

7.3. The Gate Applied Retrospectively

Table 9 applies the three checks to our project. Taken together, the limited oracle discrimination, strong rank attenuation and lack of a feature advantage over drift support a decision against further deployment of the tested design. These measurements could have preceded full framework development.

Across the 25 diagnostic instances, κ_{VND} ranges from 0.664 to 0.951, with mean 0.889. These values summarise

recorded-cohort rank–gain association in the selected runs, not an intrinsic constant of the problem or operator.

The checks can precede deployment of an online filtering framework. They require instrumented logs and offline analysis, including sufficient independent runs for model validation; one run alone cannot support run-aware cross-validation. The present application is retrospective and exploratory. A prospective study must specify its practical quality tolerance and computational costs before using these measurements to make a deployment decision.

7.4. Interpreting the Outcome

The gate is one-sided. Adverse findings across the checks can support a decision not to proceed with a particular filtering design under the assessed conditions. High κ , interpreted with weak input–output rank transmission and limited predictive value, provides evidence for that decision; high κ alone is not an impossibility result. Favourable findings permit further evaluation but do not establish net benefit. A low κ addresses only one potential obstacle, leaving target predictability, discrimination beyond drift, calibration, overhead and search-quality effects to be assessed. The gate therefore screens deployment choices rather than proving a universal boundary between feasible and infeasible filtering.

We state our own values as a single calibration point rather than as a standard, and recommend that studies of surrogate-assisted local-search filtering report κ alongside their outcome. One point does not determine a curve; a literature that reports the coefficient would.

7.5. Alternatives Suggested by the Gate

The diagnostics also suggest alternative designs to evaluate, each addressing a different potential limitation.

Consider a different target. Attenuation of solution-quality ordering does not by itself determine the predictability of local-search runtime. If runtime varies substantially and is predictable from pre-step descriptors, the architecture could support cost-aware triage. Its savings and effects on solution quality would need separate evaluation; we did not pursue this direction here.

Move the decision point. The gate evaluates one specific insertion: before the expensive step. A surrogate may still earn its place elsewhere in the loop, for example in operator or parameter selection, where the quantity being predicted is not mediated by a contraction.

Assess alternative local-search operators. The measured κ depends on the operator and the input distribution. Operators with stronger rank preservation may provide more favourable conditions for filtering, subject to the other checks and an end-to-end assessment.

8. Discussion

8.1. Scope of the Findings

Three findings define the scope of the study. The evaluated pre-local-search survival filters show no statistically detectable benefit; the diagnostic measurements identify limited discrimination and attenuation of cost ordering as plausible contributing factors; and these properties can be measured before deployment. The findings concern the tested pipeline, representations and protocol. The transferable contribution is a screening procedure whose interpretation requires evidence from each new setting.

The value of reporting it lies in that specificity. Pang et al. (2022) argue, from a different corner of evolutionary computation, that results which contradict the field's working intuitions are worth publishing precisely because they expose assumptions that otherwise remain untested, and that such results are informative only when their cause is identified rather than merely observed. The assumption exposed here is that an offspring's pre-local-search description carries usable information about its post-local-search fate. That assumption is implicit in every pre-local-search filter, including our own earlier one. Explicitly measuring it helps distinguish predictive limitations from implementation and evaluation effects.

8.2. When Can Pre-Local-Search Filtering Pay Off?

The diagnosis suggests conditions worth evaluating, consistent with the permissive interpretation of passing the gate.

A rank-preserving operator may retain information useful to a pre-step predictor. The greedy stage in our sample shows substantial preservation on two instances and weaker transmission on the third. Stronger preservation removes one concern, while leaving the relation between cost, survival and useful rejection decisions to be tested.

If the dominant expense is objective evaluation rather than an intervening improvement operator, the specific rank-transmission concern may be less relevant. Predictive accuracy and the balance between saved computation and search-quality effects would still require evaluation.

An alternative target is local-search runtime. The measured attenuation of quality ordering does not establish whether runtime is predictable. A model of expected expense could support cost-aware decisions, provided its overhead and effects on solution quality are assessed. Our logs do not record per-offspring local-search time, so this remains a direction for future work.

8.3. Relation to the Preliminary Study

The preliminary study (Balemir et al., 2026) reported that 13–26% of offspring could be pruned at negligible cost in quality. The present experiment does not provide statistical support for a benefit under the larger evaluation protocol. The shared framework makes the differences in evaluation design relevant to interpreting the two results.

Three features of the earlier design limit interpretation. It used four instances at a 500-iteration budget with five runs each, providing limited evidence for separating a filtering effect from seed variance. It had no in-run control: the filtered and unfiltered configurations were separate executions rather than concurrent islands of the same run, so the comparison absorbed any difference in machine state, and the reported figures were best-of-five rather than paired. And it compared configurations per iteration while pruning changes what an iteration costs: a filtered run performs fewer local searches per iteration by construction, so equal-iteration comparisons favour the filtered configuration.

That study also reported no concordance index for any fitted model, leaving the selectivity of its pruning unresolved. The present near-chance measurements motivate this diagnostic, but do not retrospectively determine the unmeasured concordance of the earlier runs. The gate of Section 7 is intended to make such an assessment part of the evaluation before deployment.

8.4. Threats to Validity

Unpaired seeds. Island i draws seed $\sigma + i$, so a model and the control never share a random stream and per-instance differences contain seed variance as well as treatment effect. Aggregation over 53 instances recovers power, and the signed-rank test pairs at the instance level, but a paired-seed design would have been stronger and we recommend it.

Contended timing. All ten islands occupy one machine concurrently. Wall-clock speedups are therefore measured under mutual load and should be read as total-runtime comparisons under contention, as noted in Section 4.3.

Breadth of the diagnosis. The measurements are not uniformly broad. The null result covers all 53 test instances; the contraction coefficient is measured on 25; the concordance and oracle analyses use five instances chosen to span the structural groups; and the stage-wise chain uses the three instances for which the local search was instrumented. The narrowest of these, the stage chain, is also the most mechanistic, and the wider κ measurement is consistent with it everywhere.

Selected records and incomplete cohorts. Unadmitted offspring, terminal survivors and zero-lifetime records are absent from the diagnostic sample. Reconstructing cohorts from subsets of logs also omits live members. Neither the cost rank nor the edit-distance rank in Equation (1) is exactly reconstructed. The oracle value 0.687 is therefore an empirical result on this selected sample, not a ceiling for all offspring. Correcting this selection requires recording all admission outcomes, complete population state and terminal censoring, then repeating the diagnostic analyses.

Diagnostic comparisons. Model/oracle and lineage experiments include matched time-index controls, but the exploratory diversity and partial-stage measurements do not establish incremental value beyond drift. Surpassing or matching the clock alone does not establish conditional independence or useful within-time discrimination. A time-only versus time-plus-features comparison and

within-time evaluation would be needed for that stronger claim.

Missing pruning ablation. The experiment compares learned filtering with no filtering, not with matched-rate random pruning. It therefore does not separate the value of learned selection from the effect of reducing VND calls. Such an ablation and time-to-target comparisons are needed to assess selective filtering utility more directly.

One local-search design. Every measurement concerns a single improvement pipeline. The observed rank-gain association motivates assessment in other settings, but does not establish its prevalence across improvement operators. The gate has been applied retrospectively in one system without a demonstrated successful filtering case. Prospective evaluations across operators are needed to assess its screening value, using κ alongside rank transmission and end-to-end outcomes.

Logging integrity. The parallel islands write to a shared standard output without a lock, so a small fraction of log lines interleave and yield uninterpretable iteration numbers. These are discarded by range filtering before analysis. The defect affects log presentation only, not the search itself.

9. Conclusion

We built a complete survival-based offspring filter for a state-of-the-art memetic algorithm for the CETSP: nine survival models, an adaptive training trigger grounded in events-per-variable, validation-calibrated rejection thresholds, a runtime rejection cap, a resident-process inference path that removes the cross-language latency identified as the main obstacle in earlier work, and a parallel harness that places all nine treatments and an unfiltered control in the same run. Evaluated on 53 benchmark instances with five seeds, the filter changes neither solution quality nor convergence speed to any statistically detectable degree, in either direction.

The diagnostic contribution is a set of measurements that helps interpret this outcome. The tested models show limited survival discrimination, while the tested representations show limited discrimination. The oracle and lineage comparisons provide matched iteration-index controls. A model using recorded post-local-search information reaches 0.687, an empirical reference rather than a formal upper bound. Local search attenuates entering cost ordering, with a rank correlation between recorded cohort position and full-VND gain of 0.889 averaged over 25 instances. This provides a plausible mechanism for the limited usefulness of cost-based pre-VND descriptors in the evaluated pipeline.

These findings motivate a one-sided viability gate comprising three diagnostic checks that can precede full surrogate deployment. The contraction coefficient is calibrated against cost-rank concordance on the observed data, and interpreted alongside target predictability and discrimination beyond temporal drift. Applied retrospectively, the joint evidence supports a decision against further deployment of the tested design. Passing the gate would instead justify further

evaluation; it would not establish net benefit. The procedure supplies neither a universal cut-point nor an impossibility claim for other filtering designs.

Three directions follow: assess local-search runtime as a target for cost-aware triage, including its effects on solution quality; evaluate the gate prospectively across different improvement operators; and examine surrogate decisions elsewhere in the search loop. Each requires direct evidence of predictive usefulness and end-to-end benefit.

Code, parsed results and analysis scripts accompany the paper; raw logs are available on request as detailed in the data-availability statement.

CRedit authorship contribution statement

Demir Balemir: Conceptualisation, Methodology, Software, Formal analysis, Investigation, Data curation, Visualisation, Writing – original draft. **Deniz Cantürk:** Conceptualisation, Supervision, Validation, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The solver, survival-modelling pipeline, diagnostic scripts and retained measurement CSVs are publicly available at <https://github.com/DemirBalemir/MA-CETSP-ml-optimization>, together with parsed end-to-end results. The release rebuilds tables and figures from these retained measurements and documents their evaluation protocols. Raw per-solution logs, approximately 3.4 million records, are available from the authors on request for recalculating diagnostics. The full-VND calibration includes a run manifest and input hashes; exact historical selections for every older probe are not bundled. The repository README distinguishes result-level reproduction from refitting models on the original logs.

References

- Balemir, D., Cantürk, D., & Dökeroğlu, T. (2026). New learning-assisted memetic algorithm for the close-enough traveling salesman problem (LA-CETSP). *2026 5th International Informatics and Software Engineering Conference (IISEC)*, 364–369. <https://doi.org/10.1109/IISEC69317.2026.11418468>
- Behdani, B., & Smith, J. C. (2014). An integer-programming-based approach to the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 26(3), 415–432. <https://doi.org/10.1287/ijoc.2013.0574>
- Bolufé-Röhler, A., & Tamayo-Vera, D. (2025). Machine learning for enhancing metaheuristics in global optimization: A comprehensive review. *Mathematics*, 13(18), Article 2909. <https://doi.org/10.3390/math13182909>
- Burk, L., Zolas, J., Bischl, B., Bender, A., Wright, M. N., & Sonabend, R. (2026). A large-scale neutral comparison study of survival models on low-dimensional data. *Bioinformatics*, 42(5), Article btg186. <https://doi.org/10.1093/bioinformatics/btg186>
- Cantürk, D. (2025). A new parallel fitness-oriented memetic algorithm for the close enough traveling salesman problem. *2025 33rd Signal Processing and Communications Applications Conference (SIU)*, 1–4. <https://doi.org/10.1109/SIU66497.2025.11111810>
- Cantürk, D., & Dökeroğlu, T. (2026). Scalable parallel memetic algorithms for large close-enough traveling salesman problems. *Neural Computing and Applications*, 38(2), Article 27. <https://doi.org/10.1007/s00521-025-11706-4>
- Cariou, C., Moiroux-Arvis, L., Pinet, F., & Chanet, J.-P. (2023). Evolutionary algorithm with geometrical heuristics for solving the close enough traveling salesman problem: Application to the trajectory planning of an unmanned aerial vehicle. *Algorithms*, 16(1), Article 44. <https://doi.org/10.3390/a16010044>
- Carrabs, F., Cerrone, C., Cerulli, R., & Gaudioso, M. (2017). A novel discretization scheme for the close enough traveling salesman problem. *Computers & Operations Research*, 78, 163–171. <https://doi.org/10.1016/j.cor.2016.09.003>
- Carrabs, F., Cerrone, C., Cerulli, R., & Golden, B. (2020). An adaptive heuristic approach to compute upper and lower bounds for the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 32(4), 1030–1048. <https://doi.org/10.1287/ijoc.2020.0962>
- Carrabs, F., Cerulli, R., D'Ambrosio, C., & Murano, G. (2025). Upper bound computation for the multiple close-enough traveling salesman problem. *Proceedings of the 14th International Conference on Operations Research and Enterprise Systems (ICORES)*, 186–195. <https://doi.org/10.5220/0013377900003893>
- Coutinho, W., Nascimento, R., Pessoa, A., & Subramanian, A. (2016). A branch-and-bound algorithm for the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 28(4), 752–765. <https://doi.org/10.1287/ijoc.2016.0711>
- Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2), 187–202. <https://doi.org/10.1111/j.2517-6161.1972.tb00899.x>
- Cui, W., Zhu, J., & Peng, L. (2026). On the service time in the close-enough traveling salesman problem [In press]. *European Journal of Operational Research*. <https://doi.org/10.1016/j.ejor.2026.08.001>
- Derouiche, H., Elarbi, M., & Bechikh, S. (2025). Deep crossover schemes for genetic algorithms: Investigations on the travel salesman problem. *Swarm and Evolutionary Computation*, 98, Article 102094. <https://doi.org/10.1016/j.swevo.2025.102094>
- Di Placido, A., Archetti, C., & Cerrone, C. (2022). A genetic algorithm for the close-enough traveling salesman problem with application to solar panels diagnostic reconnaissance. *Computers & Operations Research*, 145, Article 105831. <https://doi.org/10.1016/j.cor.2022.105831>
- Di Placido, A., Archetti, C., Cerrone, C., & Golden, B. (2023). The generalized close enough traveling salesman problem. *European Journal of Operational Research*, 310(3), 974–991. <https://doi.org/10.1016/j.ejor.2023.04.010>
- Dökeroğlu, T., & Cantürk, D. (2025). New Harris Hawks algorithms for the close-enough traveling salesman problem. *Intelligent Systems with Applications*, 28, Article 200586. <https://doi.org/10.1016/j.iswa.2025.200586>
- Fan, Y., Zhang, L., Xu, Y., Li, C., & Wu, X. (2025). A unified deep reinforcement learning approach for close enough traveling salesman problem. *arXiv preprint arXiv:2510.03065*. <https://doi.org/10.48550/arXiv.2510.03065>
- Gulczynski, D. J., Heath, J. W., & Price, C. C. (2006). The close enough traveling salesman problem: A discussion of several heuristics.

- In F. B. Alt, M. C. Fu, & B. L. Golden (Eds.), *Perspectives in operations research* (pp. 271–283). Springer.
https://doi.org/10.1007/978-0-387-39934-8_16
- Hanawa, Y., Harada, T., & Miura, Y. (2025). Impact of surrogate model accuracy on performance and model management strategy in surrogate-assisted evolutionary algorithms. *Array*, 27, Article 100461. <https://doi.org/10.1016/j.array.2025.100461>
- Hao, H., Zhang, X., & Zhou, A. (2024). Enhancing SAEAs with unevaluated solutions: A case study of relation model for expensive optimization. *Science China Information Sciences*, 67(2), Article 120103.
<https://doi.org/10.1007/s11432-023-3909-x>
- He, C., Zhang, Y., Gong, D., & Ji, X. (2023). A review of surrogate-assisted evolutionary algorithms for expensive optimization problems. *Expert Systems with Applications*, 217, Article 119495. <https://doi.org/10.1016/j.eswa.2022.119495>
- Hothorn, T., Bühlmann, P., Dudoit, S., Molinaro, A., & van der Laan, M. J. (2006). Survival ensembles. *Biostatistics*, 7(3), 355–373. <https://doi.org/10.1093/biostatistics/kxj011>
- Ikeguchi, T., Nishihara, K., Kawauchi, Y., Koguma, Y., & Nakata, M. (2025). A surrogate-assisted memetic algorithm for permutation-based combinatorial optimization problems. *Swarm and Evolutionary Computation*, 98, Article 102060. <https://doi.org/10.1016/j.swevo.2025.102060>
- Ishwaran, H., Kogalur, U. B., Blackstone, E. H., & Lauer, M. S. (2008). Random survival forests. *Annals of Applied Statistics*, 2(3), 841–860. <https://doi.org/10.1214/08-AOAS169>
- Ismail, A. H., Song, X., Ouelhadji, D., Al-Behadili, M., & Fraess-Ehrfeld, A. (2025). Unmanned surface vessel routing and unmanned aerial vehicle swarm scheduling for off-shore wind turbine blade inspection. *Expert Systems with Applications*, 284, Article 127534. <https://doi.org/10.1016/j.eswa.2025.127534>
- Jin, Y. (2005). A comprehensive survey of fitness approximation in evolutionary computation. *Soft Computing*, 9(1), 3–12. <https://doi.org/10.1007/s00500-003-0328-5>
- Jin, Y. (2011). Surrogate-assisted evolutionary computation: Recent advances and future challenges. *Swarm and Evolutionary Computation*, 1(2), 61–70. <https://doi.org/10.1016/j.swevo.2011.05.001>
- Kärcher, J., & Meyr, H. (2025). A machine learning approach for predicting the best heuristic for a large scaled capacitated lotsizing problem. *OR Spectrum*, 47(3), 889–931. <https://doi.org/10.1007/s00291-024-00804-9>
- Katzman, J. L., Shaham, U., Cloninger, A., Bates, J., Jiang, T., & Kluger, Y. (2018). DeepSurv: Personalized treatment recommender system using a Cox proportional hazards deep neural network. *BMC Medical Research Methodology*, 18(1), Article 24. <https://doi.org/10.1186/s12874-018-0482-1>
- Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! *International Conference on Learning Representations*. <https://openreview.net/forum?id=ByxBFsRqYm>
- Lei, Z., & Hao, J.-K. (2024). An effective memetic algorithm for the close-enough traveling salesman problem. *Applied Soft Computing*, 153, Article 111266. <https://doi.org/10.1016/j.asoc.2024.111266>
- Li, B., Yang, Y., Liu, D., Zhang, Y., Zhou, A., & Yao, X. (2024). Accelerating surrogate assisted evolutionary algorithms for expensive multi-objective optimization via explainable machine learning. *Swarm and Evolutionary Computation*, 88, Article 101610. <https://doi.org/10.1016/j.swevo.2024.101610>
- Li, J., Wang, P., Dong, H., Shen, J., & Chen, C. (2022). A classification surrogate-assisted multi-objective evolutionary algorithm for expensive optimization. *Knowledge-Based Systems*, 242, Article 108416. <https://doi.org/10.1016/j.knsys.2022.108416>
- Lim, D., Jin, Y., Ong, Y.-S., & Sendhoff, B. (2010). Generalizing surrogate-assisted evolutionary computation. *IEEE Transactions on Evolutionary Computation*, 14(3), 329–355. <https://doi.org/10.1109/TEVC.2009.2027359>
- Liu, S., Wang, H., Peng, W., & Yao, W. (2024). Surrogate-assisted evolutionary algorithms for expensive combinatorial optimization: A survey. *Complex & Intelligent Systems*, 10(4), 5933–5949. <https://doi.org/10.1007/s40747-024-01465-5>
- Pang, L. M., Ishibuchi, H., & Shang, K. (2022). Counterintuitive experimental results in evolutionary large-scale multiobjective optimization. *IEEE Transactions on Evolutionary Computation*, 26(6), 1609–1616. <https://doi.org/10.1109/TEVC.2022.3161363>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Peduzzi, P., Concato, J., Feinstein, A. R., & Holford, T. R. (1995). Importance of events per independent variable in proportional hazards regression analysis. II. Accuracy and precision of regression estimates. *Journal of Clinical Epidemiology*, 48(12), 1503–1510. [https://doi.org/10.1016/0895-4356\(95\)00048-8](https://doi.org/10.1016/0895-4356(95)00048-8)
- Riley, R. D., Snell, K. I. E., Ensor, J., Burke, D. L., Harrell, F. E., Moons, K. G. M., & Collins, G. S. (2019). Minimum sample size for developing a multivariable prediction model: Part II – binary and time-to-event outcomes. *Statistics in Medicine*, 38(7), 1276–1296. <https://doi.org/10.1002/sim.7992>
- Simon, N., Friedman, J. H., Hastie, T., & Tibshirani, R. (2011). Regularization paths for Cox’s proportional hazards model via coordinate descent. *Journal of Statistical Software*, 39(5), 1–13. <https://doi.org/10.18637/jss.v039.i05>
- Talbi, E.-G. (2021). Machine learning into metaheuristics: A survey and taxonomy. *ACM Computing Surveys*, 54(6), 1–32. <https://doi.org/10.1145/3459664>
- Van Belle, V., Pelckmans, K., Suykens, J. A. K., & Van Huffel, S. (2007). Support vector machines for survival analysis. *Proceedings of the Third International Conference on Computational Intelligence in Medicine and Healthcare (CIMED2007)*, 1–8.
- Wei, L.-J. (1992). The accelerated failure time model: A useful alternative to the Cox regression model in survival analysis. *Statistics in Medicine*, 11(14–15), 1871–1879. <https://doi.org/10.1002/sim.4780111409>
- Wiegrefe, S., Kopper, P., Sonabend, R., Bischl, B., & Bender, A. (2024). Deep learning for survival analysis: A review. *Artificial Intelligence Review*, 57(3), Article 65. <https://doi.org/10.1007/s10462-023-10681-3>
- Wu, X., Wen, L., Xiao, Y., Wu, C., Wu, Y., Yu, C., Maskell, D. L., Zhou, Y., & Wang, D. (2026). Neural combinatorial optimization algorithms for solving vehicle routing problems: A comprehensive survey with perspectives [Early access]. *IEEE Transactions on Neural Networks and Learning Systems*. <https://doi.org/10.1109/TNNLS.2026.3713193>
- Yu, C.-N., Greiner, R., Lin, H.-C., & Baracos, V. (2011). Learning patient-specific cancer survival distributions as a sequence of dependent regressors. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, & K. Weinberger (Eds.), *Advances in neural information processing systems*. Curran Associates. <https://proceedings.neurips.cc/paper/2011/hash/1019c8091693ef5c5f55970346633f92-Abstract.html>
- Zhai, Z., Tan, Y., Li, X., Li, J., & Zhang, H. (2024). A composite surrogate-assisted evolutionary algorithm for expensive many-objective optimization. *Expert Systems with Applications*, 236, Article 121374. <https://doi.org/10.1016/j.eswa.2023.121374>
- Zhang, W., Sauppe, J. J., & Jacobson, S. H. (2023). Results for the close-enough traveling salesman problem with a branch-and-bound algorithm. *Computational Optimization and*

Applications, 85(2), 369–407.

<https://doi.org/10.1007/s10589-023-00474-3>

- Zhang, W., Li, C., Gen, M., Yang, W., & Zhang, G. (2024). A multiobjective memetic algorithm with particle swarm optimization and Q-learning-based local search for energy-efficient distributed heterogeneous hybrid flow-shop scheduling problem. *Expert Systems with Applications*, 237, Article 121570. <https://doi.org/10.1016/j.eswa.2023.121570>
- Zhang, Y., Bai, R., Qu, R., Tu, C., & Jin, J. (2022). A deep reinforcement learning based hyper-heuristic for combinatorial optimisation with uncertainties. *European Journal of Operational Research*, 300(2), 418–427. <https://doi.org/10.1016/j.ejor.2021.10.032>
- Zhao, F., Liu, Y., Zhu, N., Xu, T., & Jonrinaldi. (2023). A selection hyper-heuristic algorithm with Q-learning mechanism. *Applied Soft Computing*, 147, Article 110815. <https://doi.org/10.1016/j.asoc.2023.110815>
- Zhen, L., Yang, Z., Laporte, G., Yi, W., & Fan, T. (2024). Unmanned aerial vehicle inspection routing and scheduling for engineering management. *Engineering*, 36, 223–239. <https://doi.org/10.1016/j.eng.2023.10.014>
- Zheng, N., Wang, H., & Liu, J. (2024). Formulating approximation error as noise in surrogate-assisted multi-objective evolutionary algorithm. *Swarm and Evolutionary Computation*, 90, Article 101666. <https://doi.org/10.1016/j.swevo.2024.101666>